

Workflow Blueprint: expanded technical outline

1. Preliminary tabular data preparation

a. Input data identification.

This step establishes the operational entry point of the workflow, i.e. whether the project starts from newly collected tabular data or from an already available external source. It also fixes the initial semantic and procedural assumptions under which later RDF materialisation will remain deterministic.

i. Data collection (in case of a dataset to be created from scratch).

Applies when the dataset is produced directly within the project.

- **Minimum technical inputs**
 - shared tabular workspace;
 - field-level data-entry conventions;
 - mandatory-column policy;
 - local identifier policy for row-level traceability;
 - controlled-value guidance where applicable;
 - separator/date-format conventions;
 - at least one sample row or instructional record per field.
- **Core technical operations**
 - design of reusable spreadsheet templates;
 - explicit declaration of column semantics;
 - distinction between operational templates and sample/instructional tables;
 - encoding of relation fields needed for later entity linking;
 - inclusion of contributor guidance for non-technical data entry.
- **Expected outputs**
 - structured tabular templates;
 - accompanying samples;
 - field definitions/documentation;
 - deterministic header structure for conversion.
- **Reusable implementations / artefacts**
 - CHANGES data collection templates and samples;
 - Digital Damaged Ceramics extended templates;
 - DDC JSON field-name mappings as a schema-translation layer between CHANGES-native and project-specific headers.

or

ii. Dataset selection (in case multiple access approaches are provided by a data source).

Applies when the source exposes multiple retrieval channels, such as dumps, APIs, SPARQL endpoints, or queryable services.

- **Minimum technical inputs**
 - list of available access channels;

- source assessment criteria;
- scope and temporal boundaries;
- initial sampling strategy.
- **Core technical operations**
 - comparison of access channels against scope, stability, and coverage;
 - declaration of the selected retrieval route;
 - production of an initial extraction plan;
 - controlled exploratory extraction before full-scale processing.
- **Expected outputs**
 - declared access strategy;
 - extraction scripts or query set;
 - representative exploratory sample;
 - initial monitoring and test artefacts.
- **Reusable implementations / artefacts**
 - OpenCitations Data Sources Converter as a reusable source-to-tabular crosswalk layer;
 - Early Modern BnF Data Collector and Harmoniser as a SPARQL-based retrieval and consolidation workflow.

1. Data extraction and production of a sample dataset for exploratory purposes, accompanied by monitoring and automated tests.

- **Technical function**
 - expose data structure, instability, and retrieval cost before committing to full-scale processing.
- **Outputs**
 - sample dataset;
 - extraction logs;
 - monitoring artefacts;
 - source-specific automated checks.
- **Implementation suggestions**
 - chunked processing;
 - parameterised run scripts;
 - caching where identifier validation or repeated retrieval is involved.

2. Human inspection of the sample dataset and feedback to identify harmonisation needs.

- **Technical function**
 - translate exploratory retrieval into explicit harmonisation requirements.
- **Expected outputs**
 - list of required interventions, e.g. header renaming, separator policies, value-normalisation rules, missing-field handling, disambiguation priorities, cardinality alerts.
- **Reusable implementations / artefacts**
 - BnF actor-variability diagnostics;

- column-level reconnaissance and matching analyses;
- JSON or report-based anomaly summaries.

iii. Identification of the legal and ethical constraints to the reuse of collected data.

- **Minimum technical inputs**
 - institutional policies;
 - source licensing terms;
 - rights-holder requirements;
 - dissemination constraints.
- **Expected outputs**
 - reuse policy per data class;
 - licence choices for tabular and RDF outputs;
 - restrictions affecting public dissemination, interfaces, or linked files.
- **Reusable implementations / artefacts**
 - explicit licence handling in Aldrovandi;
 - CC0 citation/provenance releases in OpenCitations;
 - version-preserving publication policies in dissemination environments.

b. Harmonisation and cleaning, accompanied by evaluation and tests.

- **Technical function**
 - produce curated tables whose structure and values satisfy the declared conversion assumptions.
- **Core technical operations**
 - deterministic header translation;
 - schema crosswalking;
 - unification of heterogeneous exports;
 - separator policies for multi-valued cells;
 - normalisation of date/value conventions;
 - deduplication and conflict detection;
 - field-cardinality analysis;
 - identifier syntax/existence validation.
- **Expected outputs**
 - curated tables;
 - transformation logs;
 - diagnostic reports;
 - residual anomaly lists.
- **Reusable implementations / artefacts**
 - DDC JSON header mappings;
 - OpenCitations identifier-validation and storage-management components;
 - BnF unifiers, name/ID matching scripts, cardinality analyses, and conflict reports.

c. Dataset enrichment with persistent identifiers and links to external sources.

- **Technical function**
 - increase linkability, reduce ambiguity, and prepare the tabular layer for semantic reconciliation.
- **Core technical operations**
 - insertion or validation of external authority identifiers;
 - creation of stable internal identifiers where needed;
 - linking to authority records and controlled vocabularies;
 - retrieval of authority-style attributes for selected entities.
- **Expected outputs**
 - enriched tables with resolvable identifiers and outbound links.
- **Reusable implementations / artefacts**
 - identifier-management modules handling DOI, ORCID, VIAF, Wikidata, etc.;
 - BnF enrichment queries over actor URIs and external matches;
 - template-level guidance recommending authority identifiers and controlled vocabularies.

d. Definition of research-oriented requirements and querying desiderata.

- **Technical function**
 - shift the dataset from a descriptive inventory to a research-oriented data product.
- **Core technical operations**
 - formalisation of the entities and relations that must be queryable;
 - declaration of required filters, aggregations, and analytical perspectives;
 - identification of the minimal provenance signals needed for result interpretation;
 - anticipation of dissemination-layer requirements.
- **Expected outputs**
 - explicit querying desiderata;
 - interface requirements;
 - modelling constraints derived from the intended research use.
- **Reusable implementations / artefacts**
 - precompiled SPARQL query sets;
 - dissemination-oriented interface requirements;
 - modelling decisions taken to support multi-perspective navigation.

i. Production of purpose-oriented optimised subsets together with traceability artefacts.

- **Technical function**
 - derive subsets when the full dataset is too broad, heterogeneous, or computationally inconvenient for a given analytical task.
- **Expected outputs**
 - purpose-oriented subsets;

- provenance logs preserving traceability to sources and transformations;
- chunked or partitioned exports for downstream analysis.
- **Reusable implementations / artefacts**
 - BnF per-year splits and chunked exports;
 - provenance logs created during unification processes.

e. Publication of the resulting tabular datasets and complementary materials with persistent identifiers under an open licence, to foster data access, trustworthiness, and research reproducibility.

- **Technical function**
 - turn the tabular layer into a stable, citable, and reusable research object before RDF generation.
 - **Expected outputs**
 - deposited tabular distributions;
 - versioning and persistent identifiers;
 - supporting documentation sufficient to reproduce the conversion;
 - publication of templates, samples, and schema-mapping files where relevant.
 - **Reusable implementations / artefacts**
 - CHANGES templates and samples release;
 - Aldrovandi CSV datasets;
 - DDC template/documentation package;
 - Zenodo deposits of mapping/configuration support materials.
-

2. RDF graph materialisation

a. RDF graph generation from curated data.

- **Technical function**
 - execute the conversion under an explicit target model and mapping strategy.
- **Minimum technical inputs**
 - curated tables;
 - target application profile or data model;
 - mapping files;
 - runtime configuration;
 - function libraries and orchestration logic.
- **Core technical operations**
 - table parsing according to declared conventions;
 - subject/object generation via mappings and functions;
 - controlled-value alignment;
 - date handling and multi-value parsing;
 - merge of graph fragments derived from distinct input tables;
 - tolerance of partial input availability where required.
- **Expected outputs**
 - RDF serialisations;

- execution diagnostics;
- mapping/configuration artefacts linked to the run.
- **Reusable implementations / artefacts**
 - Morph-KGChad stable materialisation core;
 - released INI + YARRRML package;
 - DDC Morph-KGChad fork for translated/extended templates;
 - BnF RDF graph generation scripts;
 - CHAD-ASK as a mapping/configuration generator for semi-automatic formalisation.

b. with validation

- **Technical function**
 - verify structural and semantic consistency of produced graphs.
- **Validation targets**
 - duplicate IRIs;
 - disjoint-type conflicts;
 - inconsistent temporal bounds;
 - violations of expected single-valued properties;
 - sampled external-link dereference checks.
- **Expected outputs**
 - machine-readable quality report;
 - anomaly counts and sampled cases;
 - execution parameters associated with the validation run.

c. and testing (both qualitative and quantitative, through semantic and code-based approaches, as well as human supervision and feedback cycles).

- **Technical function**
 - treat testing as a workflow-native output, not as an ad hoc post-processing activity.
- **Testing components**
 - code-level checks;
 - runtime/memory monitoring;
 - regression-oriented diagnostics;
 - human inspection of sample output graphs;
 - iterative refinement cycles linked to observed failures.
- **Reusable implementations / artefacts**
 - Morph-KGChad monitoring/performance reports;
 - harmonisation diagnostics in BnF workflows;
 - TDD-oriented practice in reusable software components.

3. RDF data dissemination

a. Publication of the RDF serialisation (e.g., TTL) citable release, through FAIR-compliant platforms.

- **Technical function**
 - stabilise the RDF dump as a citable research output.
- **Expected outputs**
 - archived RDF release;
 - persistent identifier / versioning;
 - release metadata;
 - licence aligned with reuse policy.
- **Reusable implementations / artefacts**
 - CHAD-KG TTL release;
 - Digital Damaged Ceramics RDF/CSV release;
 - OpenCitations citation and provenance distributions in multiple serialisations.

b. Publication of a SPARQL endpoint.

- **Technical function**
 - provide stable programmatic access to the graph.
- **Minimum technical inputs**
 - validated RDF dataset;
 - triplestore deployment;
 - endpoint configuration and operational documentation.
- **Expected outputs**
 - public query endpoint;
 - documented access expectations;
 - optionally complementary programmatic services.
- **Reusable implementations / artefacts**
 - public endpoints for CH knowledge graphs;
 - OpenCitations Index endpoint and service layer;
 - DDC endpoint integrated with the public site.

c. Configuration and publication of a semantic access layer for data dissemination, such as a web portal, that enables data access for both experts and general users.

- **Technical function**
 - provide mediated access without requiring SPARQL literacy.
- **Core technical operations**
 - configuration of query templates;
 - browsing and navigation views;
 - visualisation components;
 - dataset download areas;
 - integration of endpoint-backed exploration into public interfaces.
- **Expected outputs**
 - website or portal layer;
 - precompiled queries;
 - browsing and visualisation features;

- public-facing documentation.
- **Reusable implementations / artefacts**
 - DDC website toolchain and reproducibility package;
 - embedded SPARQL interface with selectable precompiled queries;
 - static publishing layers for RDF datasets.

d. Integration of a provenance management and change tracking approach.

- **Technical function**
 - preserve auditability, interpretability, and citability across dataset evolution.
 - **Expected outputs**
 - provenance graph or provenance tables;
 - change logs;
 - versioning policy preserving prior releases;
 - structured reporting of execution and validation conditions.
 - **Reusable implementations / artefacts**
 - OpenCitations provenance datasets;
 - provenance graph and preserved versions in DDC dissemination;
 - explicit provenance reporting linked to validation outputs.
-

4. Overall quality check

a. User feedback gathering.

- **Technical function**
 - capture adequacy and usability issues across templates, outputs, and access layers.
- **Expected outputs**
 - structured feedback items convertible into harmonisation tasks, mapping revisions, documentation updates, or interface corrections.

b. Qualitative and quantitative evaluation (of the process as a whole: quality of main output, as well as performance reports of the process or specifically-implemented quality assessments, and human feedback if collected, in order to understand if the process needs refinement).

- **Technical function**
 - consolidate the evidence base for evaluating both output quality and workflow adequacy.
- **Evaluation inputs**
 - validation reports;
 - runtime/performance reports;
 - statistics on outputs;
 - sample-based inspections;
 - user/stakeholder feedback where available.
- **Expected outputs**

- structured comparative assessment across runs, versions, or case studies.
- **Reusable implementations / artefacts**
 - FAIRness assessment matrix adapted for heritage collections;
 - monitoring and validation reports generated by the executable workflows.

c. Reiteration planning in case of refinement necessities or data updates.

- **Technical function**
 - formalise when and how the workflow is re-executed.
- **Expected outputs**
 - re-iteration plan specifying:
 - which phase must be rerun;
 - which artefacts can be reused unchanged;
 - which artefacts must be regenerated;
 - which deviations from previous runs must be documented.
- **Technical importance**
 - this step turns the blueprint into a maintainable lifecycle model rather than a one-off conversion procedure.